

MoFANG Hash function

Submission to ICCS NGCC

Wenquan Bi* Boxin Zhao Xiaoyang Dong Keting Jia
Lingyue Qin

Zhongguancun Laboratory, Beijing, P.R.China
Tsinghua University, Beijing, P.R.China

June 29, 2026

Abstract

The hash function is one of the most important cryptographic primitives, which provides fundamental foundation of tamper resistance and non-repudiation, making it widely adopted in financial services, digital economy and cyberspace security. In this document, we provide the specification of MoFANG, a family of hash functions based on a 576-bit block cipher, with digest length of 256, 512, 768 and 1024, as well as two versions of XOF. It is designed to achieve a balance between security and efficiency, including throughput and resource cost, while can be relatively lightweight implemented in a bit-slice fashion.

Contents

1	Introduction	4
2	Specification	4
2.1	Notations	4
2.2	Parameters	4
2.3	Hashing Mode	5
2.3.1	Davies-Meyer	5
2.3.2	Merkle-Damgård construction and MDP	6
2.3.3	Hirose Double-Block-Length Hash Function and MDPH	7
2.4	Specification of Block Cipher MOFANG-BC	9
2.4.1	Specification of S-box	10
2.4.2	Specification of MixColumn	11
2.4.3	Specification of ShiftRow	12
2.4.4	Key Expansion	12
2.5	Padding	16
2.6	Domain separation	16
2.7	Hashing	16
2.8	Extendable Output Functions	18
3	Security Claims	20
4	Features	21
5	Design Rationale	22
5.1	Design of the Modes	22
5.2	Design of the Fundamental Block Cipher	22
5.2.1	Choice of the Substitution Layer	22
5.2.2	Choice of Linear Diffusion Layer	22
6	Security Analysis	23
6.1	Theoretical Security Analysis	23
6.2	Practical Security Analysis	23
6.2.1	Practical Security of block-cipher based hash function	24
6.2.2	Basic Components Properties	25
6.2.3	Security against differential cryptanalysis	25
6.2.4	Meet-in-the-middle Attack	26
6.3	Quantum-Attack Analysis	26
6.4	Randomness Analysis	27
6.5	Length Extension Attacks	27
7	Implementation	28
7.1	Bit-slice and Parallel Implementation	28
7.1.1	S-boxes	28
7.1.2	MixColumns	28
7.1.3	ShiftRows	29

7.2	SIMD Accelerated Implementation	29
7.3	Resource-constrained Implementation	29
7.4	Software Implementation Performance	29
8	Conclusion	30
A	Table	34
B	Randomness Analysis Result	34

1 Introduction

MoFANG, or 魔方, derived from the Cube or Rubik's Cube, is a cipher suite containing hash functions and extendable output functions MoFANG-XOF based on the 576-bit block cipher MoFANG-BC. Our aim is to provide a new relatively lightweight suite of hash algorithms that performs well on common computers with relatively small costs while providing satisfactory digest lengths of 512 and 1024 bits required by the ICCS NGCC. The family of MoFANG consists of four versions of hash functions with digest length 256, 512, 768, 1024, as well as two versions of extendable output functions (XOFs) MoFANG-256-XOF and MoFANG-768-XOF. MoFANG works in two modes of operations, *i.e.*, the Davies-Meyer Hashing Mode [20] integrated with MDP [16] for digest lengths of 256/512 and MDPH [22] which is based on Hirose's Double-Block-Length Hash Function [15] and MDP for digest lengths of 768/1024.

2 Specification

2.1 Notations

r	number of rounds of round function
h	sizes of the digest in bits
k	size of key/message block length in bits
n	size of block cipher in bits
$MSB_s(x)$	the most significant s bits of x
$LSB_s(x)$	the least significant s bits of x

2.2 Parameters

The MoFANG cipher suite consists of a family of hash functions MoFANG-HASH and the extendable output functions MoFANG-XOF. The MoFANG-HASH has 4 variants of hash function with digest length $h=256, 512, 768$, and 1024 bits, denoted as (MoFANG-256, MoFANG-512, MoFANG-768, and MoFANG-1024). Two extendable output functions (XOFs) to produce hash outputs of arbitrary length are defined as MoFANG-256-XOF and MoFANG-768-XOF.

The MoFANG family can be built with n -bit-state block cipher, we call MoFANG-BC. Let $E : \{0, 1\}^n \times \{0, 1\}^k \rightarrow \{0, 1\}^n$, which denote a block cipher with n -bit block length and k -bit key length where $k = 2n$ or $3n$ in this document.

- For MoFANG-256 and MoFANG-512, the hash functions are built on the Davies-Meyer Hashing Mode [20] with the block cipher MoFANG-BC, and then integrating with the MDP construction[16]. They can be parameterized by the digest length h , the block size n , the number of rounds r of the block cipher, the input message M and a constant initial value IV . In these versions, we use block cipher with state size $n \geq h$, *i.e.*, the block cipher MoFANG-BC with $n = 576$.

- For MoFANG-768 and MoFANG-1024, the hash functions are built on MDPH[22], which is based on Hirose Double-Block-Length Hash Function [15] and MDP construction[16], with the block cipher MoFANG-BC whose size $n < h$ while $2n \geq h$. They can be parameterized by the digest length h , the state size $2n$, the number of rounds r , the input message M and a constant initial value IV .
- For MoFANG-256-XOF, which is based on the hash function MoFANG-256, the output is of arbitrary length ℓ which consists of data blocks whose size does not exceed 256 bits.
- For MoFANG-768-XOF, which is base on the hash function MoFANG-768, the output is of arbitrary length ℓ which consists of data blocks whose size does not exceed 768 bits.

Recommended Parameter. Our recommended variant of hash function is MoFANG-512 and the recommended variant of extendable output functions is MoFANG-256-XOF.

Table 1 summarizes the instances for hash functions and and XOF with specified parameters.

Table 1: Parameters for hashing functions and XOFs

Name	Modes	Bit size of			Rounds
		digest (h)	block (n)	state($n/2n$)	
MoFANG-256	DM+MDP	256	576	576(n)	16
MoFANG-512	DM+MDP	512	576	576(n)	16
MoFANG-768	MDPH	768	576	1152($2n$)	16
MoFANG-1024	MDPH	1024	576	1152($2n$)	16
MoFANG-256-XOF	DM+MDP	arbitrary	576	576(n)	16
MoFANG-768-XOF	MDPH	arbitrary	576	1152($2n$)	16

2.3 Hashing Mode

2.3.1 Davies-Meyer

Block cipher based hashing modes. The construction of hash functions based on secure block ciphers is a mature and practical approach[20]. There are several widely used hashing modes such as DM, MMO, and MP[23], which are shown in Figure 1. With these hashing modes, the block cipher BC can be converted into a compression function. For DM mode, the compression function is defined as such an iteration: $BC\text{-}DM(h_{i-1}, m_i) = h_i = BC_{h_{i-1}}(m_i) \oplus m_i$, where m_i is the message with $M = m_0 \parallel \dots \parallel m_n$. We refer h_0 as the initial value IV , and h_i with $i > 0$ as chaining values. Integrating these compression functions into some classical construction such as Merkle-Damgård construction [21, 10] results in block-cipher-based Hash functions.

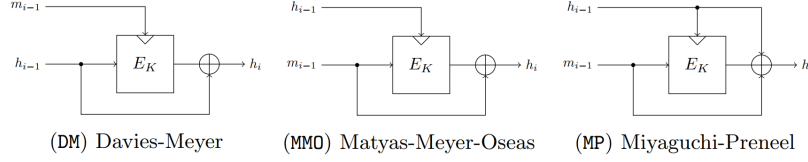


Figure 1: Common Hashing Modes

2.3.2 Merkle-Damgård construction and MDP

The **Merkle–Damgård construction** is a fundamental iterative design paradigm in modern cryptographic hash function design, formally proposed and proven independently by Ralph Merkle[21] and Ivan Damgård[10]. It serves as a standard domain extension mechanism that transforms a fixed-length collision-resistant compression function $CF : \{0, 1\}^n \times \{0, 1\}^m \rightarrow \{0, 1\}^n$ into a variable-input-length cryptographic hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^n$.

In this construction, an arbitrary-length input message M is first padded and split into a sequence of fixed-size message blocks $M = m_0 \parallel m_1 \parallel \dots \parallel m_{k-1}$. A fixed public initial value $IV \in \{0, 1\}^n$ is initialized as the starting chaining variable $h_0 = IV$. Iteratively, each chaining state is updated as follows:

$$h_{i+1} = CF(h_i, m_i), \quad \forall i \in \{0, 1, \dots, k-1\}$$

After processing all message blocks, the final chaining output h_k is taken as the hash digest $\mathcal{H}(M)$.

A core security property of the Merkle–Damgård construction is its **collision resistance preservation**: if the underlying compression function CF is collision-resistant, then the overall hash function $\mathcal{H}$ constructed by this scheme is also collision-resistant. This provable reduction laid the theoretical foundation for almost all classical mainstream hash functions, including MD5, SHA-1, and the SHA-2 family [28]. There are some inherent structural weaknesses of the Merkle–Damgård iteration, such as length-extension attacks, the multi-collision attacks.

The Merkle–Damgård construction with permutation (MDP). The *Merkle–Damgård with Permutation (MDP)*[16] is an enhanced variant of the classical Merkle–Damgård hash function construction. Compared with the traditional structure, MDP introduces an extra public permutation π to process the chaining value before the final compression iteration, which can effectively resist the length extension attack. This construction is shown in Figure 2.

Let IV be the initial value, F be the compression function, π be a fixed public permutation, and $M_1, M_2, \dots, M_n$ be the padded message blocks. The

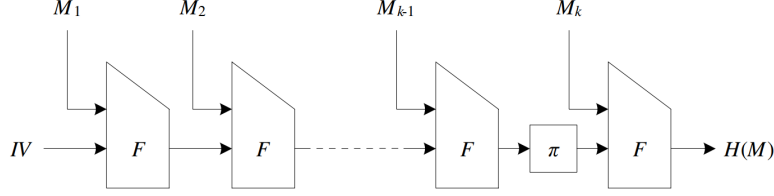


Figure 2: The structure of MDP

iterative process of MDP is defined as follows:

$$\begin{aligned} h_0 &= IV, \\ h_i &= F(h_{i-1}, M_i), \quad i = 1, 2, \dots, n-1, \\ h_n &= F(\pi(h_{n-1}), M_n), \end{aligned}$$

where the final output h_n is the hash digest of the input message.

2.3.3 Hirose Double-Block-Length Hash Function and MDPH

Hirose double-block-length (DBL) hash function, proposed by Shoichi Hirose at FSE 2006 [15], is a prominent block-cipher-based iterated hash construction designed to achieve a $2n$ -bit output using a single block cipher with $2n$ -bit keys and n -bit blocks. It belongs to the family of rate-1 DBL schemes that invoke the underlying block cipher twice per compression call, offering a favorable balance between security, efficiency, and implementation.

Let $E : \{0, 1\}^n \times \{0, 1\}^{n+b} \rightarrow \{0, 1\}^n$ be a secure block cipher with n -bit block and $n + b$ -bit key where $b > 0$, and let $c \in \{0, 1\}^n$ be a fixed non-zero constant. The compression function $H : \{0, 1\}^{2n+b} \rightarrow \{0, 1\}^{2n}$ maps a $2n$ -bit chaining state (g_{i-1}, h_{i-1}) and an b -bit message block m_i to a new $2n$ -bit state (g_i, h_i) via:

$$\begin{aligned} g_i &= E_{h_{i-1} \| m_i}(g_{i-1}) \oplus g_{i-1}, \\ h_i &= E_{h_{i-1} \| m_i}(g_{i-1} \oplus c) \oplus g_{i-1} \oplus c. \end{aligned}$$

This construction is shown in Figure 3.

The MDP with Hirose DBL (MDPH). The *MDPH*[22] is the combination of MDP with Hirose DBL, which achieves optimal indistinguishability from a random oracle, while eliminating the need for explicit post-processing steps and supporting block ciphers whose key length is longer than a single block. MDPH provides strong provable security guarantees without sacrificing efficiency or compatibility with modern block ciphers.

Let E denote a block cipher with n -bit block length and k -bit key length where $k > n$. Let $m = k - n$, the compression function $\text{Hirose}^E : \{0, 1\}^{2n} \times \{0, 1\}^m \rightarrow \{0, 1\}^{2n}$ is defined as follows.

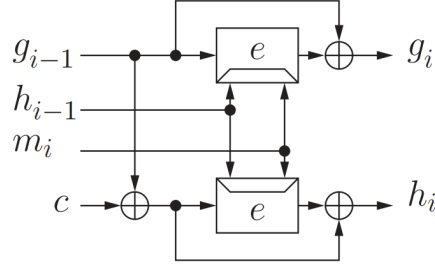


Figure 3: The compression function of Hirose DBL construction

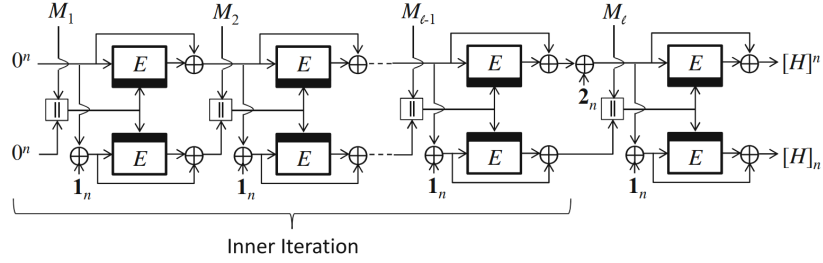


Figure 4: MDPH

$$\text{Hirose}^E(S_{i-1}, M_i) = S_i, \text{ where}$$

$$[S_i]^n = E(M_i \parallel [S_{i-1}]_n, [S_{i-1}]^n) \oplus [S_{i-1}]^n \quad (\text{Top Part})$$

$$[S_i]_n = E(M_i \parallel [S_{i-1}]_n, [S_{i-1}]^n \oplus \mathbf{1}_n) \oplus [S_{i-1}]^n \oplus \mathbf{1}_n. \quad (\text{Bottom Part})$$

Next, $\text{MDPH} : \{0, 1\}^* \rightarrow \{0, 1\}^{2n}$ is defined. Let $\text{pad} : \{0, 1\}^* \rightarrow (\{0, 1\}^m)^*$ be an injective padding function, e.g., one-zero padding $\text{pad}(M) = M \parallel 10^p$ where $p = m - 1 - |M| \bmod m$. Then, $\text{MDPH}^E(M) = H$ is defined as follows, and is illustrated in Figure 4.

1. $S_0 \leftarrow 0^{2n}$
2. $M_1, M_2, \dots, M_\ell \xleftarrow{m} \text{pad}(M)$
3. for $i = 1, \dots, \ell - 1$ do $S_i \leftarrow \text{Hirose}^E(S_{i-1}, M_i)$ // Inner Iteration
4. $[S_{\ell-1}]^n \leftarrow [S_{\ell-1}]^n \oplus \mathbf{2}_n$
5. $H \leftarrow \text{Hirose}^E(S_{\ell-1}, M_\ell)$

A distinguishing feature of MDPH is its *optimal indistinguishability security bound*, which matches the best known results for DBL hashing while avoiding final truncation or feedforward post-processing commonly used in competing designs. Furthermore, MDPH retains the simplicity of iterative hashing and avoids complex key scheduling or state expansion. Its resistance to generic attacks including collision attacks, preimage attacks, and length-extension attacks is formally proven in the ideal cipher model, making it a highly competitive candidate for lightweight and high-security hash function implementations in modern cryptographic systems.

2.4 Specification of Block Cipher MOFANG-BC

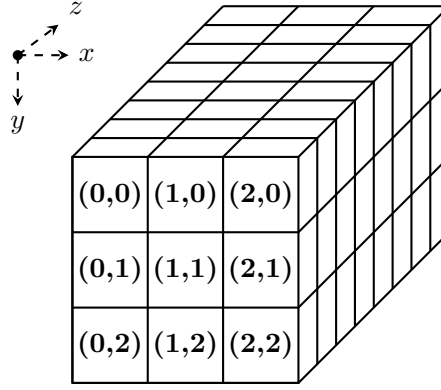


Figure 5: Parts of the State Array

The main component of MOFANG is based on the block cipher MOFANG-BC. The round function of MOFANG-BC operates on a 576-bit state $\mathcal{I}$ that is a three-dimensional array of elements of $\text{GF}(2)$, namely $\mathcal{I}[3][3][64]$. The expression $\mathcal{I}[x, y, z]$, with $0 \leq x, y \leq 2$ and $0 \leq z \leq 64$, denotes the bit in position (x, y, z) . The state array is illustrated in Fig. 5 for case $0 \leq z \leq 7$. The words are fed into the state sequentially by row. That is, $(0, 0)$ at first, then $(1, 0), (2, 0), (0, 1) \dots$. The key length is twice or three times of the state size 576-bit, *i.e.*, *MoFang-BC- $n-2n$* and *MoFang-BC- $n-3n$* .

The round function p applies four operations in sequence: the **S-box**, **Mix-Column**, **ShiftRow**, and **AddRoundKey**. The specifications of these operations are provided below.

Algorithm 1: The Block Cipher MOFANG-BC

Input: State $\mathcal{I}$, master key block key , number of rounds r

Output: State $\mathcal{I}$

```
1  $K \leftarrow key$ 
2  $A \leftarrow \mathcal{I} \oplus K$  /* Whitening key */
3 for  $i = 0$  to  $r - 1$  do
    /* key expansion, refers to Section 2.4.4 */
4      $K \leftarrow \text{KeyExp}(K)$ 
    /* S-box for state */
5      $B \leftarrow \text{SBox}(A)$ 
    /* MixColumn */
6      $C \leftarrow \text{MDS}(B)$ 
    /* ShiftRow */
7     for  $x = 0$  to 2 do
8         for  $y = 0$  to 2 do
9              $D[x, y] \leftarrow \text{RotR}(C[x, y], \rho_y^i);$ 
        /* AddRoundKey */
10     $A \leftarrow D \oplus K$ 
11  $\mathcal{I} \leftarrow A$ 
12 return  $\mathcal{I}$ 
```

2.4.1 Specification of S-box

We use a lightweight 3-bit S-box in the round function. Several 3-bit S-boxes have already been employed in the cryptographic algorithm design, the most famous is the S-box used in Simon [3] and 5-bit S-box used in Keccak [7], which could be easily changed to 3-bit S-box, as well as the 3-bit S-box used in lightweight block cipher SEA[27] and PRINTcipher[19]. we choose the 3-bit S-box same as [27] as follows:

$$S\text{-box} : x \mapsto y = S(x) \Leftrightarrow$$

$$y_0 = (x_1 \wedge x_2) \oplus x_0$$

$$y_1 = (x_2 \wedge x_0) \oplus x_1$$

$$y_2 = (x_0 \vee x_1) \oplus x_2$$

where $\wedge$ and $\vee$ respectively represent the bitwise AND and OR.

The lookup table of MOFANG's S-box is shown in Table 2.

Table 2: The lookup table of S-box

x	0	1	2	3	4	5	6	7
$\text{SB}(x)$	0	5	6	7	4	3	1	2

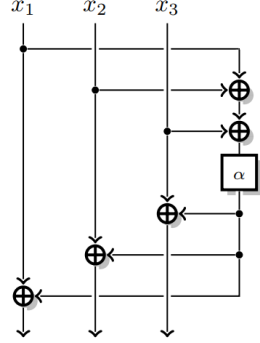


Figure 6: MDS matrix from [11]

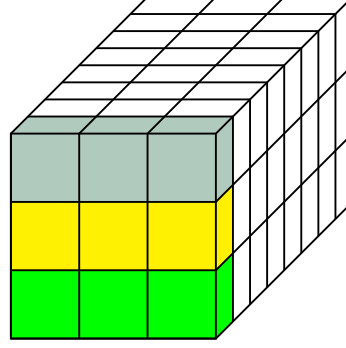


Figure 7: Illustration of MixColumn

2.4.2 Specification of MixColumn

In this step, each row is regarded as a variable in $\mathbb{F}_2^3$. For every slice, the MixColumn operation is performed by left-multiplying with an MDS matrix M , as illustrated in Fig. 7. Specifically, we have

$$\begin{bmatrix} C[\cdot, 0, z] \\ C[\cdot, 1, z] \\ C[\cdot, 2, z] \end{bmatrix} = M \cdot \begin{bmatrix} B[\cdot, 0, z] \\ B[\cdot, 1, z] \\ B[\cdot, 2, z] \end{bmatrix}.$$

We used the MDS matrix over $\text{GF}(2^3)$ from [11], which is the least costly matrix with 5 XORs plus one linear operation. The MDS matrix is as follows:

$$M = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & 2 \\ 2 & 2 & 3 \end{bmatrix}$$

The transformation can be implemented efficiently as depicted on Fig. 6. The MDS operation is given formally in Algorithm 2, where the α or MUL function is a linear equation, transforming the three bits $x_2, \dots, x_0$ of each nibble by a LFSR as shown in Algorithm 2 line 1-3.

Algorithm 2: MDS function MDS(B)

Input: B

Output: C

```

1 Function MUL( $S = (x_0, x_1, x_2)$ ):
2    $tmp \leftarrow x_2, x_2 \leftarrow x_1, x_1 \leftarrow x_0, x_0 \leftarrow tmp, x_1 \leftarrow x_1 \oplus x_0;$ 
3   return  $(x_0, x_1, x_2);$ 

4  $(S_0, S_1, S_2) \leftarrow (B[0 - 2, 0], B[0 - 2, 1], B[0 - 2, 2])$ 
5  $tmp \leftarrow S_0 \oplus S_1 \oplus S_2;$ 
6  $tmp \leftarrow \text{MUL}(tmp);$ 
7  $S_2 \leftarrow S_2 \oplus tmp;$ 
8  $S_1 \leftarrow S_1 \oplus tmp;$ 
9  $S_0 \leftarrow S_0 \oplus tmp;$ 
10  $(C[0 - 2, 0], C[0 - 2, 1], C[0 - 2, 2]) \leftarrow (S_0, S_1, S_2)$ 
11 return  $C$ 
12 end

```

2.4.3 Specification of ShiftRow

Each row is moved along the z direction by several bits, which depends on the fixed SR constants in the round i .

$$D[x, y, z] = \text{RotR}(C[x, y, z], \rho_y^i) = C[x, y, z - \rho_y^i \bmod 64], \quad 0 \leq x, y \leq 2, \quad 0 \leq z \leq 63. \quad (1)$$

The round function employs different rotation constants depending on the round number i and $0 \leq x, y \leq 2$, as shown in Table 3.

Table 3: Values of ρ_y^i used in round function i ($i \geq 0$)

(x, y)	(0, 0)	(0, 1)	(0, 2)	(1, 0)	(1, 1)	(1, 2)	(2, 0)	(2, 1)	(2, 2)
$i \bmod 2 = 0$	0	9	18	3	12	21	6	15	24
$i \bmod 2 = 1$	0	8	16	24	32	40	48	56	7

2.4.4 Key Expansion

For the key expansion of the block cipher MoFANG-BC used in the DM or MDPH hash mode, we duplicate the functions used in the round function. For simplicity, we discard the computationally heavy MDS matrix, and reuse the S-box and ShiftRows operation. The messages after padding are divided into 576-bit blocks. For each block, it first passes through the S-box layer for the first row in three, while for DBL permutation, only the second block and the third block as the first block serve as the chaining value, in fact. The first block of keys only makes a position transform for DBL construction. Meanwhile, other

blocks make a transposition of the words, followed by the ShiftRow operation. Finally, the round constant is xored to the key. In round i , the round constant xor the last byte of the first word for the last block in each round. The round constants are truncated from the value of π as a one-byte word for each round. The details of round constants are listed in Table 2.4.4. The lookup table for transposition of words is shown in table 5. As the first block is the chaining value in the DBL mode, we use the same transposition constants as the third block, except for the additional ShiftRow constants. The ShiftRow constants are shown in table 6. All round key blocks are xored together and then xor to the main state of round function.

Round i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$RC[i]$	0x24	0x3F	0x6A	0x88	0x85	0xA3	0x08	0xD3	0x13	0x19	0x8A	0x2E	0x03	0x70	0x73	0x44
Round i	16	17	18	19	20	21	22	23								
$RC[i]$	0xA4	0x09	0x38	0x22	0x29	0x9F	0x31	0xD0								

Table 4: Round constants derived from the hexadecimal digits of π .

Table 5: Values of transposition for words used in message expansion

(x, y)	(0, 0)	(0, 1)	(0, 2)	(1, 0)	(1, 1)	(1, 2)	(2, 0)	(2, 1)	(2, 2)
first block	(0, 0)	(2, 0)	(1, 0)	(1, 1)	(0, 1)	(2, 1)	(2, 2)	(1, 2)	(0, 2)
second block	(0, 0)	(1, 1)	(2, 2)	(0, 2)	(1, 0)	(2, 1)	(0, 1)	(1, 2)	(2, 0)
third block	(0, 0)	(2, 0)	(1, 0)	(1, 1)	(0, 1)	(2, 1)	(2, 2)	(1, 2)	(0, 2)

Table 6: Values of ρ_i used in key expansion

(x, y)	(0, 0)	(0, 1)	(0, 2)	(1, 0)	(1, 1)	(1, 2)	(2, 0)	(2, 1)	(2, 2)
ρ_1	1	17	41	5	0	47	11	31	59
ρ_2	3	43	0	7	53	29	19	61	37

For MOFANG-256 and MOFANG-512, two 576-bit block messages are fed into the compression function state in the form of a master key. For MOFANG-768 and MOFANG-1024, we need a longer message to injected into the compression function state, so we make three 576-bit blocks as master key. They use different parameters, first xored with each other, and then xored with the main round state. The message expansion operations are given formally in Algorithms 3 and 4 .

Algorithm 3: The Key Expansion For MOFANG-256/512

Input: State M_1, M_2 (input length: $576 * 2$)
Output: State K

```
1  $A \leftarrow M_1$ 
  /* S-box for the first row */
2  $B[0] \leftarrow \text{SBox}(A[0])$ 
3 for  $x = 1$  to 2 do
4    $B[x] \leftarrow A[x]$ 
  /* Transposition of words for the first block */
5 for  $x = 0$  to 2 do
6   for  $y = 0$  to 2 do
7      $C[x, y] \leftarrow B[y, 2x + y]$ 
  /* ShiftRow for the first block */
8 for  $x = 0$  to 2 do
9   for  $y = 0$  to 2 do
10     $D[x, y] \leftarrow \text{RotR}(C[x, y], \rho_1(x, y))$ 
11  $K \leftarrow D$ 
12  $A \leftarrow M_2$ 
  /* S-box for the first row */
13  $B[0] \leftarrow \text{SBox}(A[0])$ 
14 for  $x = 1$  to 2 do
15    $B[x] \leftarrow A[x]$ 
  /* Transposition of words for the second block */
16 for  $x = 0$  to 2 do
17   for  $y = 0$  to 2 do
18      $C[x, y] \leftarrow B[x + 2y, y]$ 
  /* ShiftRow for the second block */
19 for  $x = 0$  to 2 do
20   for  $y = 0$  to 2 do
21      $D[x, y] \leftarrow \text{RotR}(C[x, y], \rho_2(x, y))$ 
  /* AddConstant and xor state */
22  $D \leftarrow D \oplus RC[i]$ 
23  $K \leftarrow K \oplus D$ 
24 return  $K$ 
```

Algorithm 4: The Key Expansion for MOFANG-768/1024

Input: State M_1, M_2, M_3 (input length: $576 * 3$)**Output:** State K

```
1  $A \leftarrow M_1$ 
  /* Transposition of words for the first block */
2 for  $x = 0$  to 2 do
3   for  $y = 0$  to 2 do
4      $B[x, y] \leftarrow A[y, 2x + y]$ 
5  $K \leftarrow B$ 
6  $A \leftarrow M_2$ 
  /* S-box for the first row */
7  $B[0] \leftarrow \text{SBox}(A[0])$ 
8 for  $x = 1$  to 2 do
9    $B[x] \leftarrow A[x]$ 
  /* Transposition of words for the second block */
10 for  $x = 0$  to 2 do
11   for  $y = 0$  to 2 do
12      $C[x, y] \leftarrow B[x + 2y, x]$ 
  /* ShiftRow for the second block */
13 for  $x = 0$  to 2 do
14   for  $y = 0$  to 2 do
15      $D[x, y] \leftarrow \text{RotR}(C[x, y], \rho_1(x, y))$ 
16  $K \leftarrow K \oplus D$ 
17  $A \leftarrow M_3$ 
  /* S-box for the first row */
18  $B[0] \leftarrow \text{SBox}(A[0])$ 
19 for  $x = 1$  to 2 do
20    $B[x] \leftarrow A[x]$ 
  /* Transposition of words for the third block */
21 for  $x = 0$  to 2 do
22   for  $y = 0$  to 2 do
23      $B[x, y] \leftarrow A[y, 2x + y]$ 
  /* ShiftRow for the third block */
24 for  $x = 0$  to 2 do
25   for  $y = 0$  to 2 do
26      $D[x, y] \leftarrow \text{RotR}(B[x, y], \rho_2(x, y))$ 
  /* AddConstant and xor state */
27  $D \leftarrow D \oplus RC[i]$ 
28  $K \leftarrow K \oplus D$ 
29 return  $K$ 
```

2.5 Padding

The message M is padded to be a multiple of 576 bits. Suppose that the length of the message M is ℓ bits. Append the bit “1” to the end of the message, followed by a number of zero bits, making the padded message length an integer multiple of 576 bits.

2.6 Domain separation

We use the last four bits in IV to make the domain separation. Hence, the effective bit length for IV is left $576 - 4 = 572$ bits at most which is larger than 512. The domain encoding for different versions is listed in Table 7. Different versions have different domain decode numbers fed in the last four bits of IV . For example, the input IV value is a 572 bit number at most, which defaults to 0, concatenating 4 bits 0101 for MoFANG-256, while for MoFANG-256-XOF, the four bit number is 0111. The domain encoding should be different from the last permutation in MDP construction to avoid losing its functionality.

Table 7: Domain encoding for different hash and XOF versions

Versions	MoFANG -256	MoFANG -512	MoFANG -768	MoFANG -1024	MoFANG- 256-XOF	MoFANG- 768-XOF
Domain values	0101	0110	1010	1011	0111	1111

2.7 Hashing

We use two different compression function modes for different digest length versions, while using the same hash mode with MDP. The key component for all versions is the same, *i.e.*, the 576-bit block cipher MoFANG-BC. For digest length shorter than 576-bit versions, *i.e.*, MoFANG-256 and MoFANG-512, we use the Davies-Meyer modes[20] to form the compression function and then combine with MDP [16] to make hash constructions. In these versions, one block cipher call is sufficient for the compression function. When generating the message digest, the algorithm outputs the most significant 256/512 bits of the final internal chaining state.

For the hash versions with digest lengths larger than 576-bit, *i.e.*, MoFANG-768 and MoFANG-1024, one block cipher call is not enough, so we adopt the MDPH construction [22], combined by Hirose’s DBL compression function[15] and the MDP hash mode. By invoking the same 576-bit block cipher twice in a compression function, its output length can be twice as large as the block cipher state. Meanwhile, it is also feasible to instantiate two parallel block ciphers in hardware implementation, as a design trade-off for substantially improved throughput. When generating the message digest, the compression function

outputs the most significant 384/512 bits of each permutation block, which are concatenated to construct the output value of 768/1024-bits. The hash functions are given formally in Algorithms 5 and 6 .

Algorithm 5: The hash function MOFANG-256/512(M, ℓ)

Input: Initial Value $IV \in \{0, 1\}^\ell$ (default 0), Message $M \in \{0, 1\}^*$,
output bitsize $\ell = 256/512$ or arbitrary if $\ell = 0$
Output: Digest $T \in \{0, 1\}^\ell$

```

/* Initialization */
1 if  $\ell = 256$  then
2    $\mathcal{I} \leftarrow IV || 0^{316} || 0101$ 
3 if  $\ell = 512$  then
4    $\mathcal{I} \leftarrow IV || 0^{60} || 0110$ 
5 if  $\ell \neq 256/512$  then
6    $\mathcal{I} \leftarrow IV || 0^{316} || 0111$ 

/* Compression Function */
7  $CF \leftarrow MoFang-BC-DM(P, K)$ 
/* message parsing */
8  $M_1, \dots, M_s \leftarrow M || 1 || 0^*$ , where the  $M_i$  ( $1 \leq i \leq s$ ) is 1152 bits
/* Compression */
9 for  $i = 1$  to  $s - 1$  do
10    $\mathcal{I} \leftarrow CF(\mathcal{I}, M_i)$ 
11  $\mathcal{I} \leftarrow CF(\mathcal{I} \oplus 1_n, M_s)$ 
12  $T \leftarrow MSB_\ell(\mathcal{I})$  /* for MOFANG-256-XOF,  $\ell = 256$  if  $\ell > 256$  */
/* Output for MOFANG-256-XOF */
13 if  $\ell \neq 256/512$  then
14    $\lambda \leftarrow \lfloor \frac{\ell}{256} \rfloor$ 
15   for  $i = 1$  to  $\lambda$  do
16      $\mathcal{I} \leftarrow CF(\mathcal{I} \oplus 1_n, 0)$ 
17      $t \leftarrow (i = \lambda) ? \ell \bmod 256 : 256$ 
18      $T \leftarrow T || MSB_t(\mathcal{I})$ 

/* Output */
19 return  $T$ 
```

Algorithm 6: The hash function MOFANG-768/1024(M, ℓ)

Input: Initial Value $IV_L, IV_R \in \{0, 1\}^{\ell/2}$ (default all 0), Message $M \in \{0, 1\}^*$, output bitsize $\ell = 768/1024$ or arbitrary if $\ell = 0$
Output: Digest $T \in \{0, 1\}^\ell$

/ Compression Function */*

1 **Function** CF(L, R, M):

2 $T \leftarrow R \| M$;

3 $L' \leftarrow \text{MoFang-BC-DM}(L, T) \oplus L$;

4 $R' \leftarrow \text{MoFang-BC-DM}(L \oplus 1_n, T) \oplus L \oplus 1_n$

5 **return** (L', R');

/ Initialization */*

6 **if** $\ell = 768$ **then**

7 $L \leftarrow IV_L \| 0^{188} \| 1010, R \leftarrow IV_R \| 0^{192}$;

8 **if** $\ell = 1024$ **then**

9 $L \leftarrow IV_L \| 0^{60} \| 1011, R \leftarrow IV_R \| 0^{64}$;

10 **if** $\ell \neq 768/1024$ **then**

11 $L \leftarrow IV_L \| 0^{188} \| 1111, R \leftarrow IV_R \| 0^{192}$;

/ message parsing */*

12 $M_1, \dots, M_s \leftarrow M \| 1 \| 0^*$, where the M_i ($1 \leq i \leq s$) is 1152 bits

/ Compression */*

13 **for** $i = 1$ **to** $s - 1$ **do**

14 $(L, R) \leftarrow \text{CF}(L, R, M_i)$

15 $(L, R) \leftarrow \text{CF}(L \oplus 2_n, R, M_s)$

/ for MOFANG-768-XOF, $\ell = 768$ if $\ell > 768$ */*

16 $T \leftarrow \text{MSB}_{\frac{\ell}{2}}(L) \| \text{MSB}_{\frac{\ell}{2}}(R)$

/ Output for MOFANG-768-XOF */*

17 **if** $\ell \neq 768/1024$ **then**

18 $\lambda \leftarrow \lfloor \frac{\ell}{768} \rfloor$

19 **for** $i = 1$ **to** λ **do**

20 $(L, R) \leftarrow \text{CF}(L \oplus 2_n, R, 0)$

21 $t \leftarrow (i = \lambda) ? \ell \bmod 384 : 384$

22 $T \leftarrow T \| \text{MSB}_t(L) \| \text{MSB}_t(R)$

/ Output */*

23 **return** T

2.8 Extendable Output Functions

We build the MOFANG-XOF with an extension construction of MDP and MOFANG hash function, whose digest length is 256 or 768. We name this construction *Merkle-Damgård with Multiple Permutations (MDMP)*. Since the output length of the hash function MOFANG-256 and MOFANG-768 is much smaller than the size of its internal compression function state, intuitively a sufficient

security margin can be guaranteed during the continue iteration when the input message is all zeros. These bits, which are not output in the internal state, behave like the capacity part of a sponge structure[6]. This reserved internal state offers an inherent security margin and allows the permutation to continue iterating securely in the absence of additional incoming message input. The XOF descriptions are the same as the hash function MOFANG-256 and MOFANG-768, except for the last output part of Algorithm 5 and 6, *i.e.*, the line 13-18 in Algorithm 5 as well as the line 17-22 in Algorithm 6.

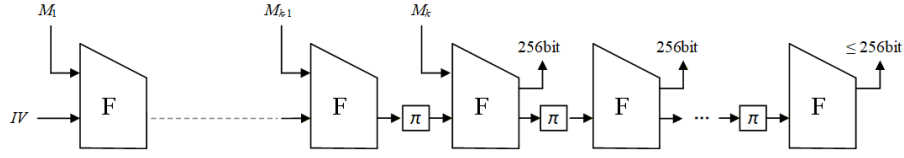


Figure 8: MDMP construction

The construction of MDMP is shown in the Figure 8. In the following, we formally provide a security proof for this construction.

Proofs. Without loss of generality, we suppose that the compression function is an ideal primitive, and the MDP construction inherits the security of MD construction such as collision resistance, (second-) preimage resistance, etc. We need to prove that when used as XOFs, the original last iteration with permutation is iterated multiple times, but it does not reduce the security level of the original construction. We prove this by induction, taking the collision resistance of 256-bit digest version MOFANG-256 as an illustration.

When the output length of the XOF is shorter than or equal to the digest length of the hash function MOFANG-256, the MOFANG-256-XOF only needs to produce output only once. In this situation, the MDMP construction is just the same as the MDP construction except for possibly truncating the output. The collision resistant strength is $\min(128, \ell/2)$ and preimage resistant strength is $\min(256, \ell)$ where $0 < \ell \leq 256$ is the output length, which inherits the MDP construction.

When the output length is two blocks of digest length, that is, $256 < \ell \leq 512$. There would be another iteration of the compression function with permutation except the origin one, but without message input controllable by the adversary. If the adversary could find a collision for these two blocks of output, thus he can use the same colliding messages to derive one block of digest collision for the original hash function. Since the one block of digest, which is truncated from the two-blocks of output of XOF, is part of the XOF output. And there are no messages input any more. This contradicts the collision-resistance property of MDP construction. It is worth noticing that outputting all state bits from the last iteration does not compromise the security of the MDP construction. While as long as this condition holds, the subsequent multiple iterations with permutation can proceed continuously. So the MDMP is collision resistant for two block output length, we arrive at an identical conclusion when the block

size is n .

The proof method for preimage resistance is identical. If the adversary could find a preimage for longer output of XOFs, he would obtain a shorter digest for hash function with the same preimage when there is no more input interference. The proof is finished.

3 Security Claims

We provide our security claims for the different variants of MoFANG in Table 8. Both MoFANG-HASH and MoFANG-XOF provide corresponding security against collision attacks and (second) pre-image attacks. Basically, for all versions of MoFANG, we claim that it is hard to find a collision with substantially less than $2^{h/2}$ hash evaluations, and hard to find a (second-)preimage with substantially less than 2^h hash evaluations, where h denotes the digest size.

The MoFANG-HASH family adopts the MDP and MDPH constructions. Compared with the original Merkle–Damgård design, the MDP construction mitigates inherent structural weaknesses such as length extension attack, multi-collision attacks, and herding attacks when the compression function is secure. The MDPH also provides strong provable security guarantees in collision attacks, preimage attacks, and length-extension attacks. It also achieves optimal indistinguishability from a random oracle with $(n - \log n)$ bit security margin. As noted in [14], Hirose’s DBL has been proved to achieve $(n - 2)$ bit collision security [15] and roughly $(2n - 5)$ bit (2nd) preimage security [1] where n is the block cipher size. The MDP structure does not weaken the security of the original structure. In contrary, it inherits all the provable security properties of the original structure such as collision resistance. Since we use a 576-bit block cipher, the collision security margin is roughly $576 - 2 = 574$, which is a security margin for largest digest where $h/2 = 512$, it’s similar for the preimage security.

For MoFANG-XOF, after the last iteration with permutation, it starts to output the random bits of the first block. In all subsequent output processes, the iteration is performed with a zero(no) input and the same compression function, enabling the output of random bits of arbitrary length. As there are a lot of reserved internal state bits that are not output in each iteration, like the capacity in Sponge. For MoFANG-XOF-256, this part is $576 - 256 = 320$ bit, while $1152 - 768 = 384$ bit for MoFANG-XOF-768. There is enough security margin to forgery the output bits. We refer to this construction as MDMP, and give a security proof of collision resistance and preimage resistance for it.

Table 8: Security claims for recommended parameters of MoFANG

Requirement	Security in bits					
	MoFANG-256	MoFANG-512	MoFANG-768	MoFANG-1024	MoFANG-256-XOF	MoFANG-768-XOF
Collision	128	256	384	512	$\min(128, \ell/2)$	$\min(384, \ell/2)$
Preimage	256	512	768	1024	$\min(256, \ell)$	$\min(768, \ell)$
2nd preimage	256	512	768	1024	$\min(256, \ell)$	$\min(768, \ell)$

4 Features

The key feature of MOFANG is its relatively small state size, resulting in low computational resource consumption and implementation cost, while maintaining efficient performance and strong security when supporting message digest lengths of 512 and 1024 bits at the same time. As the required digest length is relatively large, this imposes higher requirements on the design and security of the algorithm. There are many mature and popular hash mode constructions to design hash functions such as MD construction, wide-pipe, sponge etc. Unfortunately, 1024-bit digest needs a really large state to produce the digest in these constructions, especially for the sponge construction, which requires that the capacity size is twice as large as the digest size. It would be an enormous state. Thus the feed-forward sponge construction is another option[29, 13]. However, double-block-length compression function is a better choice since it is very flexible when implementation, and there have been already many mature studies in this field.

High security margin. The MDP and MDPH construction both have many mature security research and rigorous security proofs in terms of collision resistance, (second-preimage) resistance and resistance against length-extension attacks. As to the permutation used in MOFANG family, it adopts the operational structure like AES, and bit-level operations. Its relatively large state also ensures a high security margin.

Performance. MOFANG leverages the relatively large key size of block cipher to absorb more messages, and allows for highly parallel implementations across multiple modules including operations or permutations. In modern computers, it can be optimized because of the bit-sliced and parallelized operation design style and the relatively small permutation state in software and hardware implementation.

Flexibility. MOFANG has a large flexibility. Compared to its large digest length, MOFANG has a relatively lightweight state size and bit-sliced parallelized operations. It also would be easier to implement in embedded devices and IoT things. It is easier to achieve efficiency and resource cost balance. For double block length compression functions, it could implement two block ciphers in parallel or call the same block cipher twice. At the same time, MOFANG provides two Extendable Output Functions MOFANG-256-XOF and MOFANG-768-XOF. As far as we know, this is the first time constructing XOFs with MDMP construction.

Innovativeness. In this document, we provide MOFANG family containing four versions of hash functions and two versions of extendable output functions(XOFs) based on MDP constructions. We use a 576-bit relatively lightweight block cipher to construct the hash functions that support message digest lengths

of 512 and 1024 bits at the same time. Also, benefiting from the large redundant state, we construct XOFs based on the MDMP structure for the first time, as far as we know. In our analysis, those reserved internal bits that are not output behave like the capacity part of a sponge structure. This reserved internal state offers an inherent forgery security margin, enabling the permutation to continue iterating securely in the absence of additional message input. We give a security proof of collision resistance and preimage resistance for this construction.

5 Design Rationale

5.1 Design of the Modes

MOFANG adopts the MDP and MDPH constructions, both have mature security research and rigorous security proofs in terms of collision resistance, (second-)preimage resistance and resistance against length-extension attacks. MDP construction preserves collision resistance, random oracle and unforgeability etc. of MD construction with negligible overhead. As required by ICCS NGCC, digest lengths of 512 and 1024 bits need to be supported at the same time. So employing the same permutation function to design two algorithms is a highly reasonable choice. The larger digest version uses the double-block-length compression function to produce a double-length output from the one-block permutation. Meanwhile, these modes have rigorous security proofs already.

5.2 Design of the Fundamental Block Cipher

The fundamental block cipher follows the AES permutation’s style, this type has a lot of application and research for decades. The permutation also adopts a bit-sliced design fashion to keep a better performance and flexibility.

5.2.1 Choice of the Substitution Layer

MOFANG adopts a 3-bit S-box since the operations is less in the algebraic normal form (ANF) and bit-sliced implementation. There are several 3-bit S-box have already been employed in the cryptographic algorithm design like S-box used in Simon [3] and 5-bit S-box used in Keccak [7], which could be changed to 3-bit S-box easily, as well as the 3-bit S-box used in lightweight block cipher SEA[27] and PRINTcipher[19]. We choose a 3-bit S-box which has good differential and linear properties while with less operations and good performance such as critical path at the same time.

5.2.2 Choice of Linear Diffusion Layer

The MDS matrix has the largest branch number to achieve optimal diffusion in the SPN constructions. Meanwhile, 3×3 MDS matrix also have less operations in the algebraic normal form (ANF) and bit-sliced implementation. We choose a

lightweight MDS matrix which could be optimized to nearly optimal operations number with the least 5 XORs and 1 linear operation.

To maximize resistance against differential and linear cryptanalysis, we would like a faster diffusion speed for active bits. The internal state is defined as $3 \times 3 \times 64 = 576$ bits block in total. When one active bit after sbox and MDS matrix, it would diffuse to 9 bits. After shiftrows operation, the original one active bit could diffuse to 81 bits for 9 different shift parameters in the next round, thus it achieves full diffusion within 3 round at least. To achieve this goal, we devised two distinct sets of shift parameters across two consecutive rounds to achieve a full diffusion, one single active bit propagates to full state bits after 2.5 rounds (without the shiftrow operation for the third round).

To enhance the diffusion for the key/message expansion module, while maintaining a good performance, we apply S-box operation for the first row only in three. Then perform a transposition operation for nine words, make sure that words from the same row are permuted to distinct rows, and words from the same column are permuted to distinct columns. The shiftrow parameters for key/message expansion do not impose such strict requirements such as full diffusion. Nevertheless, the shift offsets of the key and the state should be kept as separate as possible to avoid collisions, so we adopt a set of prime numbers to serve as shift parameters. We verified that the key also achieves a fast diffusion when ignoring the reduction in S-box operations.

6 Security Analysis

6.1 Theoretical Security Analysis

The MDP and MDPH constructions both have mature security research and rigorous security proofs in terms of collision resistance, (second-)preimage resistance and resistance against length-extension attacks. So all we need is an ideal compression function used in the Merkle–Damgård construction with permutation. This problem reduces to the security of the block cipher algorithm that we used to construct the compression function. As MD, MDP, MDPH construction and DM mode all have theoretical security proofs, so we only need to make practical security analysis in terms collision attacks, (second-)preimage attacks etc. for block-cipher-based compression function. In short, the security of block cipher. In the following, we analyze practical security for the MoFANG-BC based compression function.

6.2 Practical Security Analysis

The compression function is constructed based on the block cipher MoFANG-BC with the DM mode and Hirose DBL construction, which are mature approaches. The security thereof is ultimately reduced to the security of block ciphers. In DM mode, the message is injected in the position of key, it is similar for the Hirose DBL construction. Furthermore, in the DM mode, the compression

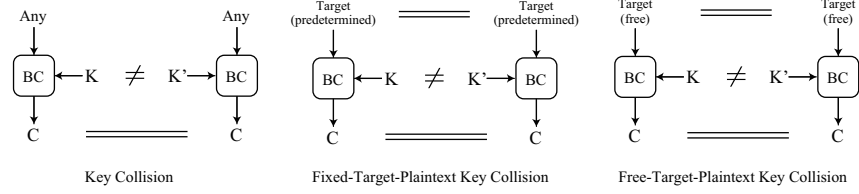


Figure 9: Variants of key collisions

function is built with the *MoFang-BC- $n-2n$* , the key length is double of the block length. While in the Hirose DBL, the key length is three times of block length, i.e., *MoFang-BC- $n-3n$* . However, the first block in the Hirose DBL construction behaves as the chaining value in fact, which cannot be controlled by the attacker, and two blocks of messages are compressed in each iteration as well. So, the most important is the practical security of the block cipher *MoFang-BC- $n-2n$* for our scheme.

6.2.1 Practical Security of block-cipher based hash function

Collision attacks. In order to find two different messages that result in an identical output for the Davies-Meyer compression function and the double-block-length compression function, it means to find the key collisions for the foundational block cipher MOFANG-BC, i.e., for the fixed-target setting, to find different K_1 and K_2 such that $\text{MOFANG-BC}_{K_1}(0) = \text{MOFANG-BC}_{K_2}(0)$ ultimately, or for the semi-free start setting, to find K_1 , K_2 and h_{i-1} such that $\text{MOFANG-BC}_{K_1}(h_{i-1}) = \text{MOFANG-BC}_{K_2}(h_{i-1})$. The main difference between them is the degree of freedom in the plaintext as shown in figure 9. To obtain a key collision, high-probability related-key differential trails are required, and there should be no active difference in the plaintext P and the ciphertext C . All active differences should be introduced by the key K in the first round and eliminated them before the last round.

Preimage attacks. For the Davies-Meyer compression function and the double-block-length compression function, the messages are fed into the compression function via the key position. Finding the preimage of key for a block cipher implies that we need to break the secret key through plaintext and ciphertext. It is a classical cryptanalytic problem. The most widely adopted cryptanalytic approaches are differential and linear cryptanalysis as well as the meet-in-the-middle attack.

Second-preimage attacks. The property of collision resistance implies that the hash function satisfies second-preimage resistance. In order to find the second-preimage message for the particular identical output, we have less degree of freedom in ciphertext compared to the situation of the fixed-target key

collision attacks, we should use a fixed ciphertext, so we lose this part of degree of freedom.

Other Security Properties. For other security properties, like partial-collision attacks, semi-free-start collision attacks, free-start collision attacks, etc., there are some differences compared to the key collision attacks mainly in the degree of freedom aspect for plaintext or ciphertext. But the most important is all similar, high-probability related-key differential trails are required, and there should be no active differences in the P and C . The longer the related-key differential characteristic, the lower the complexity, and the more effective the attack.

In summary, the security of the underlying block cipher MOFANG-BC determines the practical security of MOFANG hash family. We list some practical cryptanalysis of MOFANG-BC in the following.

6.2.2 Basic Components Properties

In this section, we give some known properties of the S-box used in MOFANG. Tab. 11 shows the differential distribution corresponding to input and output differences. As can be seen in the table, the maximum differential probability of the S-box is 2^{-2} . Tab. 12 shows the biases of the linear approximation defined by the corresponding input and output masks. The maximum linear bias is $\pm 2^{-2}$. The differential and linear distributions are optimal and have the same probability for different input and output pairs.

Let x_0, x_1, x_2 and y_0, y_1, y_2 be the 3-bit input and output of the S-box, where x_0 refers to the most significant bit of the S-box. Then the algebraic normal form (ANF) of the S-box is given by:

$$\begin{aligned} y_0 &= (x_1 \wedge x_2) \oplus x_0 \\ y_1 &= (x_2 \wedge x_0) \oplus x_1 \\ y_2 &= (x_0 \vee x_1) \oplus x_2 \end{aligned}$$

The algebraic degree of the S-box is 2.

6.2.3 Security against differential cryptanalysis

As the differential probability of the S-box is all same, 2^{-2} for one active S-box, the number of active S-box is essential for differential attacks. However, the bit-level differential trials search is a difficult task due to the large state. The design of MOFANG-BC is bit sliced, and the s-boxs would be recombined in the next round. We try to search the optimal bit-level differential trials with MILP, it takes dozens of hours to get an optimal 3-round differential trials on a desktop, while no optimal 4-round results on a server for over one week. Since the searching time increases exponentially with the number of rounds, it

seems impractical to obtain optimal differential paths in the conventional MILP method.

To nevertheless gain insight into the diffusion properties of the round function, we adopted a greedy search strategy. Starting from low-weight input differences, we iteratively construct trails by optimizing, at each round, the number of active S-boxes. While this approach does not guarantee optimality, it provides meaningful lower bounds on the number of active S-boxes over multiple rounds. Using this method, the best 9-round trails we identified for 576-bit MOFANG-BC involve at least 574 active S-boxes, the differential probability reaches 2^{-1148} . Based on this, we estimate the lower bound of active S-boxes along the optimal differential path over 9 rounds to be approximately between 300 and 400, with the differential probability lower than 2^{-576} . For DBL construction, the active s-boxes would propagate along two parallel trails. These results strongly suggest that any differential characteristic over a sufficient number of rounds must involve a very large number of active S-boxes. Consequently, it seems highly unlikely that differential attacks could lead to collisions more efficiently than generic attacks.

6.2.4 Meet-in-the-middle Attack

MOFANG-BC achieves full diffusion with three rounds, which guarantees that any bit non-linearly affects all bits after 3 rounds in the forward and the backward directions. From its application to the SPN structure [24], the number of attacked rounds can be evaluated by considering the maximum length of three features, partial-matching, initial structure and splice-and-cut, which is also adopted by the self-evaluation of SKINNY [4] and MIDORI [2]. Partial-matching [25] cannot work if the number of rounds reaches full diffusion rounds in each of forward and backward directions, the possible number of rounds for the partial matching is estimated as $(3-1)+(3-1)+1=5$ rounds. The length of the initial structure can also be bounded by the smaller number of full diffusion rounds in backwards and forwards and the maximum number that all key cells impact to an Sbox, it works up to $3+2-1=4$ rounds. The splice-and-cut may extend the number of attack rounds up to the smaller number of full diffusion rounds minus one, i.e., $3-1=2$ rounds. Then, possible number of rounds used for MitM is $5+4+2=11$ rounds, leaving a security margin of $16-11=5$ rounds for MOFANG-BC.

6.3 Quantum-Attack Analysis

Given a hash function with h -bit digest, the quantum preimage or 2nd preimage attacks should be not efficient than the Grover's algorithm [12], whose time complexities is $2^{h/2}$. At CRYPTO 2022, Schrottenloher and Stevens Marc[26] proposed the quantum preimage attack by introducing the quantum MitM attack. The quantum attack leverages the valid MitM trails. However, as analyzed in Sect. 6.2.4, no MitM trails cover the full rounds of MOFANG-BC. Therefore, there is no quantum MitM preimage attack on full MOFANG-BC.

The complexity of generic quantum collision-finding algorithms is highly dependent on the resources available to the adversary. Existing literature typically considers three scenarios, distinguished by the type and scale of memory and computing resources, each leading to different time–space tradeoffs.

- **Scenario 1.** Assuming a polynomial-size quantum computer with access to exponentially large quantum random access memory (qRAM), the BHT algorithm [8] achieves both time and qRAM complexity $2^{h/3}$. While this setting yields optimal time complexity for generic quantum collision finding, the requirement for large-scale qRAM poses significant practical challenges.
- **Scenario 2.** In the absence of exponentially large quantum or classical memory, the quantum analogue of the parallel rho method [30, 5, 17] employs S quantum processors to achieve a time-space tradeoff of $2^{h/2}/S$, making it suitable for memory-constrained settings with parallel quantum resources.
- **Scenario 3.** Allowing large classical memory while avoiding exponentially large qRAM, the CNS algorithm [9] finds collisions in time $2^{2h/5}$ using $2^{h/5}$ classical RAM, providing a balanced tradeoff between efficiency and resource availability.

According to our analysis in Sect. 6.2.3, there is no full-round differential trials which beat any of the three generic quantum collision attacks.

6.4 Randomness Analysis

We evaluate the randomness of the outputs using 15 statistical tests, including *MonoBitFrequencyTest*, *FrequencyWithinBlockTest*, *PokerTest*, *OverlappingTemplateMatchingTest*, *RunsTest*, *RunsDistributionTest*, *LongestRunOfOnesInABlockTest*, *BinaryDerivativeTest*, *AutocorrelationTest*, *MatrixRankTest*, *CumulativeTest*, *ApproximateEntropyTest*, *LinearComplexityTest*, *MaurerUniversalTest*, and *DiscreteFourierTransformTest*. For each test, the significance level is set to $\alpha = 0.01$. A sequence is considered to pass a test if the corresponding p -value is greater than or equal to α .

Our experimental results show that, for all input types (random, high Hamming weight, and low Hamming weight), the outputs consistently satisfy this criterion across all tests. Consequently, the outputs are shown to pass all statistical randomness tests. Some results of randomness analysis are listed in Section B.

6.5 Length Extension Attacks

Since we employ two hash constructions, namely the MDP combined the DM mode compression function and the MDPH structure which is the combination of Hirose DBL with MDP, both of them have the same construction MDP.

By introducing a permutation, MDP can resist length-extension attacks at an extremely low cost. Therefore, MOFANG is resistant to length extension attacks.

7 Implementation

7.1 Bit-slice and Parallel Implementation

MOFANG family is designed in a bit-slice and parallel fashion. All of the functions can be implemented in parallel. $192(64 * 3)$ S-boxes and 64 Mixcolumns operations can be executed independently. The following functions are used in the bit-slice implementation of MOFANG.

7.1.1 S-boxes

Let each $x_i (0 \leq i \leq 2)$ denote a 64-bit word, the 3-bit S-box outputs (y_0, y_1, y_2) are computed by inputs (x_0, x_1, x_2) as follows:

1. $y_0 = (x_1 \wedge x_2) \oplus x_0,$
2. $y_1 = (x_2 \wedge x_0) \oplus x_1,$
3. $y_2 = (x_0 \vee x_1) \oplus x_2.$

Here, 64 Sboxes are computed in parallel, and the Sboxes in other row could run concurrently with no mutual interference at the same time.

7.1.2 MixColumns

The bit-slice implementation of MixColumns originates from [11]. The implementation is shown in Fig. 6. The matrix multiplication can be implemented in parallel of the state, so 64 MixColumns could be executed concurrently. The bit-slice implementation with C macro fashion is shown in Listing 1, where the inputs/outputs (x_i) are 64-bit words.

Listing 1: bit-slice MDS implementation

```

1 #define MUL(x0, x1, x2, tmp) {
2     tmp = x2; x2 = x1; x1 = x0; x0 = tmp; /* rotation */
3     x1 ^= x0;                               /* XOR */
4 }
5
6 #define MDS(x0, x1, x2, x3, x4, x5, x6, x7, x8,
7     tmp, y0, y1, y2) {
8     y0 = x0 ^ x3 ^ x6;
9     y1 = x1 ^ x4 ^ x7;
10    y2 = x2 ^ x5 ^ x8; /* Y = A ^ B ^ C */
11    MUL(y0, y1, y2, tmp); /* Y = MUL(Y) */
12    x0 ^= y0; x1 ^= y1; x2 ^= y2; /* A ^= Y */
13    x3 ^= y0; x4 ^= y1; x5 ^= y2 /* B ^= Y */
14    x6 ^= y0; x7 ^= y1; x8 ^= y2 /* C ^= Y */
15 }

```

7.1.3 ShiftRows

In ShiftRow operation, a 64-bit word is shifted circularly to the left by a fixed constant. These constants are shown in Table 3. The ShiftRows operation can be computed in parallel for 9 words with the function as follows:

$$y_i = x_i \lll c_i$$

where c_i is a constant from Table 3.

7.2 SIMD Accelerated Implementation

MoFANG family is designed for mainstream 64-bit computers with relatively lightweight states when supporting larger digest. The states are represented by 64-bit words normally. However, when the vector instruction set or SIMD vector instructions are available, the states can also be stored and computed in vector registers especially for the DBL constructions. For MoFANG-768 and MoFANG-1024, two identical block ciphers are induced. The operations and computation parameters are exactly the same except the input data, which is an ideal scenario for SIMD instructions. In this situation, SIMD accelerated implementation is trivial, two 64-bit words in same position for two blocks combined to a 128-bit word, and the units perform the same operations as before. In addition, other operations such as S-boxes could be accelerated by SIMD instructions in a dedicated way. All bits occupying the same position among the three with respect to the S-box can be stored in a single large vector register, and interact with the other two registers.

7.3 Resource-constrained Implementation

In resource-constrained environments, the situation is similar to SIMD implementation, since MoFANG is designed in a bit-slice fashion. On 32-bit or 16-bit processors, a 64-bit word can be represented as two 32-bit words or four 16-bit words. As the S-box and Mixcolumn operation are performed in parallel naturally, the computation proceeds in nearly the same manner. The only extra concern is the shiftrow operation, where a 64-bit number is split into two words to perform shifting on 32-bit processors, which is already many researches such as [18] and not a very complex and slowly change. Other computations remain the same as the popular 64-bit processors except that.

7.4 Software Implementation Performance

We evaluate the software performance of MoFANG on a server equipped with Intel(R) Xeon(R) Gold 6548N processors, with a maximum CPU frequency of 4.10 GHz. The operating system is Ubuntu 22.04 LTS. The implementation is single-threaded and compiled by GCC 11.4.0 with the `-O3` optimization option. Reference implementation and three instruction-set variants are tested: SSE,

Table 9: Software performance for 1000-byte messages on the Intel Xeon Gold 6548N platform. Throughput is given in MB/s.

Name	REF	SSE	AVX2	AVX-512
MoFANG-256	773.51	764.25	759.69	749.74
MoFANG-512	827.85	755.88	786.07	818.52
MoFANG-768	294.13	446.75	456.59	630.78
MoFANG-1024	287.24	456.64	462.58	635.92
SHA3-256	-	334.71	359.21	524.85
SHA3-512	-	201.00	207.07	290.51

Table 10: Software performance for 100 MB messages on the Intel Xeon Gold 6548N platform. Throughput is given in MB/s.

Name	REF	SSE	AVX2	AVX-512
MoFANG-256	619.95	618.51	607.02	608.66
MoFANG-512	617.99	618.42	617.18	616.25
MoFANG-768	266.52	402.84	406.81	526.59
MoFANG-1024	265.11	402.10	406.89	526.10
SHA3-256	-	414.04	412.55	622.89
SHA3-512	-	219.00	214.62	307.50

AVX2, and AVX-512. For comparison, we also evaluate SHA3-256 and SHA3-512 using the implementation from XKCP under the same platform and testing methodology.

For each parameter set, two types of messages are tested. The first test uses 1000-byte messages repeated 1,000,000 times (Table 9), and the second test uses 100 MB messages repeated 10 times (Table 10). Therefore, the total amount of processed data is 1000 MB in both tests. The average throughput is computed as the total number of processed bytes divided by the elapsed time.

For MoFANG-256 and MoFANG-512, the reference implementation already delivers excellent performance, while for MoFANG-768 and MoFANG-1024, the SIMD-accelerated implementation yields substantial speedups.

8 Conclusion

In this document, we proposed MoFANG family containing hash functions and extendable output functions, which are both software and hardware efficient and flexible. When supporting message digest lengths of 512 and 1024 bits, MoFANG keeps a relatively small states based on a 576-bit block cipher MoFANG-BC. At the same time, to support XOF MDMP construction is provided. Our analysis shows that MoFANG is very secure. However, the extensive security analysis of any new design requires a lot of efforts from many researchers. We welcome and encourage researchers to analyze the security of MoFANG.

References

- [1] Frederik Armknecht, Ewan Fleischmann, Matthias Krause, Jooyoung Lee, Martijn Stam, and John Steinberger. The preimage security of double-block-length compression functions. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 233–251. Springer, 2011.
- [2] Subhadeep Banik, Andrey Bogdanov, Takanori Isobe, Kyoji Shibutani, Harunaga Hiwatari, Toru Akishita, and Francesco Regazzoni. Midori: A block cipher for low energy. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 411–436. Springer, 2015.
- [3] Ray Beaulieu, Douglas Shors, Jason Smith, Stefan Treatman-Clark, Bryan Weeks, and Louis Wingers. The simon and speck lightweight block ciphers. In *Proceedings of the 52nd annual design automation conference*, pages 1–6, 2015.
- [4] Christof Beierle, J  r  my Jean, Stefan K  lbl, Gregor Leander, Amir Moradi, Thomas Peyrin, Yu Sasaki, Pascal Sasdrich, and Siang Meng Sim. The skinny family of block ciphers and its low-latency variant mantis. In *Annual international cryptology conference*, pages 123–153. Springer, 2016.
- [5] Daniel J Bernstein. Cost analysis of hash collisions: Will quantum computers make sharcs obsolete. *SHARCS*, 9:105, 2009.
- [6] Guido Bertoni, Joan Daemen, Micha  l Peeters, and Gilles Van Assche. Sponge functions. In *ECRYPT hash workshop*, volume 2007, 2007.
- [7] Guido Bertoni, Joan Daemen, Micha  l Peeters, and Gilles Van Assche. Keccak. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 313–314. Springer, 2013.
- [8] Gilles Brassard, Peter H  yer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN 1998*, pages 163–169. Springer, 1998.
- [9] Andr   Chailloux, Mar  a Naya-Plasencia, and Andr   Schrottenloher. An efficient quantum collision search algorithm and implications on symmetric cryptography. In *ASIACRYPT 2017*, volume 10625, pages 211–240. Springer, 2017.
- [10] Ivan Bjerre Damg  rd. A design principle for hash functions. In *Conference on the Theory and Application of Cryptology*, pages 416–427. Springer, 1989.
- [11] S  bastien Duval and Ga  tan Leurent. MDS matrices with lightweight circuits. In *IACR Transactions on Symmetric Cryptology*, pages 48–78. Springer, 2018.

- [12] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.
- [13] Chun Guo, Kai Hu, Yanhong Fan, Yong Fu, and Meiqin Wang. Adding feeding forward back to the sponge construction. *Cryptology ePrint Archive*, Paper 2025/1006, 2025.
- [14] Chun Guo, Tetsu Iwata, Mustafa Khairallah, Kazuhiko Minematsu, and Thomas Peyrin. Romulus v1. 3. *Submission to NIST Lightweight Cryptography*, 2021.
- [15] Shoichi Hirose. Some plausible constructions of double-block-length hash functions. In *International Workshop on Fast Software Encryption*, pages 210–225. Springer, 2006.
- [16] Shoichi Hirose, Je Hong Park, and Aaram Yun. A simple variant of the merkle–damgård scheme with a permutation. *Journal of cryptology*, 25(2):271–309, 2012.
- [17] Akinori Hosoyamada and Yu Sasaki. Finding hash collisions with quantum computers by using differential trails with smaller probability than birthday bound. In *EUROCRYPT 2020*, pages 249–279. Springer, 2020.
- [18] Junhao Huang, Alexandre Adomnicăi, Jipeng Zhang, Wangchen Dai, Yao Liu, Ray CC Cheung, Çetin Kaya Koç, and Donglong Chen. Revisiting keccak and dilithium implementations on armv7-m. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):1–24, 2024.
- [19] Lars Knudsen, Gregor Leander, Axel Poschmann, and Matthew JB Robshaw. Printcipher: a block cipher for ic-printing. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 16–32. Springer, 2010.
- [20] J Menezes Alfred, C van Oorschot Paul, and A Vanstone Scott. Handbook of applied cryptography. *Handbook of Applied Cryptography*, 1996.
- [21] Ralph C Merkle. A certified digital signature. In *Conference on the Theory and Application of Cryptology*, pages 218–238. Springer, 1989.
- [22] Yusuke Naito. Optimally indifferentiable double-block-length hashing without post-processing and with support for longer key than single block. In *International Conference on Cryptology and Information Security in Latin America*, pages 65–85. Springer, 2019.
- [23] Bart Preneel, René Govaerts, and Joos Vandewalle. Hash functions based on block ciphers: A synthetic approach. In *Annual international cryptology conference*, pages 368–378. Springer, 1993.

- [24] Yu Sasaki. Meet-in-the-middle preimage attacks on aes hashing modes and an application to whirlpool. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 96(1):121–130, 2013.
- [25] Yu Sasaki and Kazumaro Aoki. Finding preimages in full md5 faster than exhaustive search. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 134–152. Springer, 2009.
- [26] André Schrottenloher and Marc Stevens. Simplified mitm modeling for permutations: New (quantum) attacks. In *Annual International Cryptology Conference*, pages 717–747. Springer, 2022.
- [27] François-Xavier Standaert, Gilles Piret, Neil Gershenfeld, and Jean-Jacques Quisquater. Sea: A scalable encryption algorithm for small embedded applications. In *International conference on smart card research and advanced applications*, pages 222–236. Springer, 2006.
- [28] Secure Hash Standard. Secure hash standard. *FIPS PUB*, pages 180–1, 1995.
- [29] Siwei Sun, Shun Li, Zhiyu Zhang, Charlotte Lefevre, Bart Mennink, Zhen Qin, and Dengguo Feng. Permutation-based hashing with stronger (second) preimage resistance - application to hash-based signature schemes. *Cryptology ePrint Archive*, Paper 2025/963, 2025.
- [30] Paul C Van Oorschot and Michael J Wiener. Parallel collision search with cryptanalytic applications. *Journal of cryptology*, 12(1):1–28, 1999.

Supplementary Material

A Table

Table 11: The differential distribution table of the MoFANG S-box.

	0	1	2	3	4	5	6	7
0	8	0	0	0	0	0	0	0
1	0	2	0	2	0	2	0	2
2	0	2	2	0	0	2	2	0
3	0	0	2	2	0	0	2	2
4	0	0	0	0	2	2	2	2
5	0	2	0	2	2	0	2	0
6	0	2	2	0	2	0	0	2
7	0	0	2	2	2	2	0	0

Table 12: The linear approximation table of the MoFANG S-box.

	0	1	2	3	4	5	6	7
0	4	0	0	0	0	0	0	0
1	0	2	2	0	0	-2	2	0
2	0	0	2	2	0	0	-2	2
3	0	2	0	2	0	2	0	-2
4	0	0	0	0	-2	2	2	2
5	0	2	-2	0	2	0	0	2
6	0	0	2	-2	2	2	0	0
7	0	-2	0	2	2	0	2	0

B Randomness Analysis Result

Table 13: GMT 0005-2021 Randomness Test Results of MoFANG-512 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	989	11
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	990	10
<i>PokerTest</i> ($m = 4$)	989	11
<i>PokerTest</i> ($m = 8$)	992	8
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	986	14
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	993	7
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	996	4
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	989	11
<i>RunsTest</i>	988	12
<i>RunsDistributionTest</i>	995	5
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	989	11
<i>BinaryDerivativeTest</i> ($k = 3$)	988	12
<i>BinaryDerivativeTest</i> ($k = 7$)	994	6
<i>AutocorrelationTest</i> ($d = 1$)	995	5
<i>AutocorrelationTest</i> ($d = 2$)	996	4
<i>AutocorrelationTest</i> ($d = 8$)	991	9
<i>AutocorrelationTest</i> ($d = 16$)	989	11
<i>MatrixRankTest</i>	989	11
<i>CumulativeTest</i> (Forward)	992	8
<i>CumulativeTest</i> (Backward)	990	10
<i>ApproximateEntropyTest</i> ($m = 2$)	993	7
<i>ApproximateEntropyTest</i> ($m = 5$)	993	7
<i>LinearComplexityTest</i> ($m = 500$)	987	13
<i>LinearComplexityTest</i> ($m = 1000$)	993	7
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	984	16
<i>DiscreteFourierTransformTest</i>	989	11

Table 14: GMT 0005-2021 Randomness Test Results of MoFANG-1024 with fully random input

Test Item	Passed Samples	Failed Samples
<i>MonoBitFrequencyTest</i>	988	12
<i>FrequencyWithinBlockTest</i> ($m = 10000$)	993	7
<i>PokerTest</i> ($m = 4$)	994	6
<i>PokerTest</i> ($m = 8$)	989	11
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P1)	990	10
<i>OverlappingTemplateMatchingTest</i> ($m = 3$, P2)	985	15
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P1)	994	6
<i>OverlappingTemplateMatchingTest</i> ($m = 5$, P2)	992	8
<i>RunsTest</i>	992	8
<i>RunsDistributionTest</i>	994	6
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 1-run)	988	12
<i>LongestRunOfOnesInABlockTest</i> ($m = 10000$, 0-run)	989	11
<i>BinaryDerivativeTest</i> ($k = 3$)	985	15
<i>BinaryDerivativeTest</i> ($k = 7$)	993	7
<i>AutocorrelationTest</i> ($d = 1$)	992	8
<i>AutocorrelationTest</i> ($d = 2$)	985	15
<i>AutocorrelationTest</i> ($d = 8$)	992	8
<i>AutocorrelationTest</i> ($d = 16$)	992	8
<i>MatrixRankTest</i>	990	10
<i>CumulativeTest</i> (Forward)	987	13
<i>CumulativeTest</i> (Backward)	988	12
<i>ApproximateEntropyTest</i> ($m = 2$)	990	10
<i>ApproximateEntropyTest</i> ($m = 5$)	991	9
<i>LinearComplexityTest</i> ($m = 500$)	995	5
<i>LinearComplexityTest</i> ($m = 1000$)	984	16
<i>MaurerUniversalTest</i> ($L = 7$, $Q = 1280$)	984	16
<i>DiscreteFourierTransformTest</i>	993	7